# Bene, come integrare docker compose in ddev per eseguire il progetto precedente eventualmente insieme ad altri servizi ?

Integrerei Compose in `Ddev` come **motore dichiarativo per lo stack**, mantenendo `Ddev` come interfaccia e generatore di configurazione. Il container di sviluppo resta effimero (`--rm`); Compose gestisce invece servizi opzionali — database, Redis, message broker, dev server — con reti, volumi, healthcheck e profili. Compose è pensato proprio per definire servizi, reti e volumi in un unico file YAML.[1] [2]

### Architettura consigliata

Non far generare a `Ddev` un `docker run` per ogni servizio. Fai in modo che produca o selezioni un Compose project:

```
Python/foo/
├── .ddev/
│   ├── compose.yaml            # generato o fornito dal progetto
│   ├── compose.dev.yaml        # override di sviluppo
│   └── env/dev.env
├── build.sh
├── run.sh
└── src/
```

`Ddev --target=Python/foo` diventa:

1. Determina immagine, nome progetto, UID/GID e volumi.
2. Genera o valida `.ddev/compose.yaml`.
3. Esegue `docker compose -p dev.python.foo ... up`.
4. Entra nel servizio `dev` con `docker compose exec` oppure lancia un comando one-shot con `docker compose run --rm`.

Il `--project-name` esplicito evita che Compose derivi il nome dalla directory e ti permette di conservare la convenzione `dev.python.foo`. I project name Compose devono essere limitati a lettere minuscole, cifre, trattini e underscore.[3]

### Compose base

Un esempio per un progetto Python con PostgreSQL e Redis:

```
# .ddev/compose.yaml
name: dev-python-foo

services:
  dev:
    image: dev.python
    init: true
    stdin_open: true
    tty: true
    working_dir: /workspace
    user: "${DDEV_UID}:${DDEV_GID}"
    environment:
      HOME: /home/dev
      DATABASE_URL: postgres://app:app@db:5432/app
      REDIS_URL: redis://cache:6379/0
    volumes:
      - ../:/workspace
      - venv:/home/dev/.venv
      - ./entrypoint.sh:/usr/local/bin/entrypoint.sh:ro
      - ./.bashrc:/home/dev/.bashrc:ro
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_started
    command: ["sleep", "infinity"]

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: app
    volumes:
      - db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d app"]
      interval: 5s
      timeout: 3s
      retries: 10

  cache:
    image: redis:7
    command: ["redis-server", "--save", "", "--appendonly", "no"]

volumes:
  venv:
    name: venv.dev.python.foo
  db-data:
    name: db.dev.python.foo
```

Qui `venv.dev.python.foo` mantiene esattamente il ruolo che ha già nel tuo `volumes.sh`: sopravvive alla ricreazione del container `dev`. I volumi Compose sono persistenti e indipendenti dal ciclo di vita dei container.[4]

### Override di sviluppo

Non mettere tutto in un unico file. Usa un file base riutilizzabile e un override per sviluppo; Compose carica automaticamente `compose.override.yaml` accanto al file principale, oppure puoi passare più file con `-f`, dove l'ultimo prevale in caso di conflitto.[5] [6]

```
# .ddev/compose.dev.yaml
services:
  dev:
    volumes:
      - ../:/workspace:cached
    environment:
      DEBUG: "1"
      PYTHONDONTWRITEBYTECODE: "1"

  db:
    ports:
      - "127.0.0.1:5432:5432"

  cache:
    ports:
      - "127.0.0.1:6379:6379"
```

Esporre le porte solo su `127.0.0.1` è preferibile in locale: eviti di pubblicare PostgreSQL o Redis su tutte le interfacce della macchina.

### Profili per servizi opzionali

Per non avviare sempre tutto, usa i profili. I servizi con `profiles` partono solo quando il profilo è attivato; quelli senza `profiles` partono di default.[7]

```
services:
  dev:
    # servizio principale, sempre attivo

  db:
    profiles: ["db", "full"]

  cache:
    profiles: ["cache", "full"]

  mailpit:
    image: axllent/mailpit
    profiles: ["mail", "full"]
    ports:
      - "127.0.0.1:8025:8025"
```

Poi `Ddev` può mappare flag espliciti:

```
Ddev --target=Python/foo
Ddev --target=Python/foo --with=db,cache
Ddev --target=Python/foo --profile=full
```

Internamente:

```
docker compose \
  -p dev.python.foo \
  -f .ddev/compose.yaml \
  -f .ddev/compose.dev.yaml \
  --profile db --profile cache \
  up -d
```

### Come integrarlo in `Ddev`

### 1. Generazione dichiarativa

`Ddev` può generare il Compose file dai tuoi template esistenti:

- `Dockerfile` + build args → `build:` oppure `image:`
- `entrypoint.sh` → `entrypoint:` o `command:`
- `.bashrc` → bind mount read-only
- `volumes.sh` → sezione `volumes:` con nomi deterministici
- nome directory → `name:` e `container_name`/nome servizio
- UID/GID host → `user:` e variabili d'ambiente

Consiglio però di **non rigenerare ciecamente** un file se l'utente l'ha modificato. Due modalità utili:

```
Ddev --target=Python/foo            # usa/genera se assente
Ddev --target=Python/foo --regen    # rigenera i file Compose
Ddev --target=Python/foo --check    # valida senza avviare
```

### 2. Comandi Ddev

| Comando | Operazione Compose |
|---|---|
| `Ddev --target=Python/foo` | `up -d` dei servizi richiesti, poi shell nel servizio `dev` |
| `Ddev --target=Python/foo --build` | `build` dell'immagine `dev.python` |
| `Ddev --target=Python/foo --exec="pytest"` | `run --rm dev pytest` |
| `Ddev --target=Python/foo --stop` | `stop` |
| `Ddev --target=Python/foo --down` | `down` |

| Comando | Operazione Compose |
| --- | --- |
| `Ddev --target=Python/foo --clean` | `down -v` oppure rimozione selettiva dei volumi |
| `Ddev --target=Python/foo --logs` | `logs -f dev` |

Per comandi una-tantum usa:

```
docker compose -p dev.python.foo run --rm dev pytest
```

Così ottieni lo stesso comportamento effimero del tuo attuale `--rm -it`, ma con rete, dipendenze e volumi già dichiarati.

### 3. Shell interattiva

Dopo `up -d`:

```
docker compose -p dev.python.foo exec dev bash
```

Se vuoi conservare il comportamento attuale "una shell, poi il container muore", usa invece:

```
docker compose -p dev.python.foo run --rm dev bash
```

La differenza è importante:

- `up -d` + `exec`: servizi e container `dev` restano in esecuzione.
- `run --rm`: crea un container temporaneo per il comando, poi lo rimuove; ideale per build, test e task una-tantum.

### Healthcheck e dipendenze

`depends_on` da solo stabilisce l'ordine di avvio, ma non garantisce che il servizio sia pronto. Per un database, usa `healthcheck` e `condition: service_healthy`; Compose attende che il check riesca prima di avviare il servizio dipendente. [8] [3]

```
depends_on:
  db:
    condition: service_healthy
```

Per Redis, Memcached o servizi senza un check affidabile, puoi usare `service_started`, oppure implementare un retry nel codice/entrypoint.

### Volumi: cosa delegare a Compose

| Tipo di dato | Meccanismo consigliato |
|---|---|
| Sorgenti del progetto | Bind mount della directory host |
| `venv`, `node_modules`, `target/`, cache | Volume Compose nominato |
| Dati PostgreSQL/MySQL/MongoDB | Volume Compose nominato |
| Configurazioni immutabili | Bind mount `:ro` |
| Log che vuoi conservare | Bind mount su directory host o volume dedicato |
| Segreti locali | `.env` locale escluso da Git, oppure Docker secrets |

Mantieni la convenzione dei nomi:

```
venv.dev.python.foo
db.dev.python.foo
cache.dev.python.foo
```

Così `volumes.sh` può continuare a occuparsi di creazione, permessi e ownership, mentre Compose fa riferimento ai volumi con `external: true` oppure li dichiara con `name:`.

### Volumi esterni

Se vuoi che `volumes.sh` resti il proprietario della logica di permessi:

```
volumes:
  venv:
    name: venv.dev.python.foo
    external: true
```

In questo caso Compose non crea né rimuove il volume: lo usa. È la scelta più coerente con il tuo design attuale. Se invece dichiari il volume direttamente in Compose senza `external: true`, Compose lo crea e lo gestisce, ma dovrai spostare lì anche la logica di ownership.

### Cosa evitare

- **Non usare** `container_name` **fisso** se potresti eseguire più copie dello stesso progetto: il project name Compose già namespacea container, reti e volumi.
- **Non montare l'intera home del container** come volume: nasconde file dell'immagine e rende l'ambiente meno riproducibile.
- **Non usare** `restart: always` per un ambiente di sviluppo locale, salvo casi particolari: dopo un riavvio del daemon potresti ritrovarti servizi attivi inaspettatamente.
- **Non mettere credenziali reali in** `compose.yaml`: usa `.env` locale, escluso da Git, oppure secret manager.

- **Non confondere** `down` **e** `down -v`: il primo rimuove container e reti; il secondo può rimuovere anche i volumi dichiarati nel progetto, cancellando database e cache.

### Modello operativo finale

Il flusso più pulito per il tuo caso sarebbe:

```
# Sviluppo interattivo con db e cache
Ddev --target=Python/foo --with=db,cache

# Build/test senza shell persistente
Ddev --target=Python/foo --run ./build.sh
Ddev --target=Python/foo --run pytest

# Solo servizi, senza aprire una shell
Ddev --target=Python/foo --services-up=db,cache

# Arresto pulito
Ddev --target=Python/foo --down
```

In pratica: `Ddev` **resta il front-end e il generatore di convenzioni; Compose diventa il backend dichiarativo.** Il container di sviluppo può continuare a essere monouso, mentre database, cache e altri servizi hanno un ciclo di vita separato e gestibile. [2] [1]

⁂


1. https://docs.docker.com/compose/
2. https://docs.docker.com/reference/compose-file/
3. https://github.com/compose-spec/compose-spec/blob/master/spec.md
4. https://dev.to/alexmercedcoder/a-deep-dive-into-docker-compose-27h5
5. https://cubepath.com/docs/docker-kubernetes/docker-compose-complete-guide-with-examples
6. https://dockerhol.com/blog/docker-compose-advanced-scaling-depends-on-env-files
7. https://docs.docker.com/compose/how-tos/profiles/
8. https://docs.docker.com/reference/compose-file/services/
9. https://last9.io/blog/docker-compose-health-checks/
10. https://reintech.io/blog/customize-docker-compose-extension-fields-overrides
11. https://github.com/balena-solutions/multiple-compose-files
12. https://oneuptime.com/blog/post/2025-11-27-manage-docker-compose-profiles/view
13. https://blog.ktz.me/monolithic-docker-compose-files-filtered-by-labels/
14. https://stackoverflow.com/questions/77042471/how-to-run-multiple-docker-compose-files-at-once
15. https://dokploy.com/blog/what-is-docker-compose